Agile Development Process and Devops Lab

NAME: DHINESH.S
REG NO: 23MIS0396
SET NO: 2

1 .Creation of File in GitHub Repository:

HTML File

main docker-example / index.html

Dhinesh-s2006 Update index.html

Code Blame 51 lines (42 loc) · 990 Bytes

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My Resume</title>

    <style>
        body {
            text-align: center;
            font-family: "Times New Roman", Times, serif;
            background-color: burlywood;
            margin: 0;
            padding: 0;
        }

        h1 {
            color: #0ef777;
        }

        h2 {
            color: #ffffff;
        }

        p {
            color: #ffffff;
            font-size: 18px;
        }
    </style>
</head>

<body>
    <h1>Dhinesh.S</h1>
    <p>Email: dhinesh.s2023@vitstudent.ac.in</p>
    <p>Phone Number: 1234567890</p>


    <h2>Education</h2>
```

YAML File

main docker-example / deployment.yaml

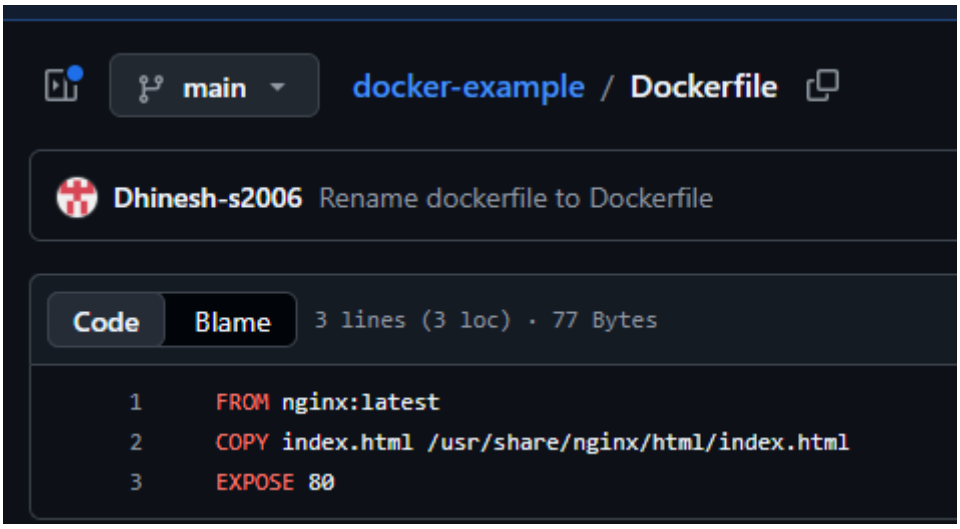
Dhinesh-s2006 Rename demployment.yaml to deployment.yaml

Code Blame 33 lines (32 loc) · 538 Bytes

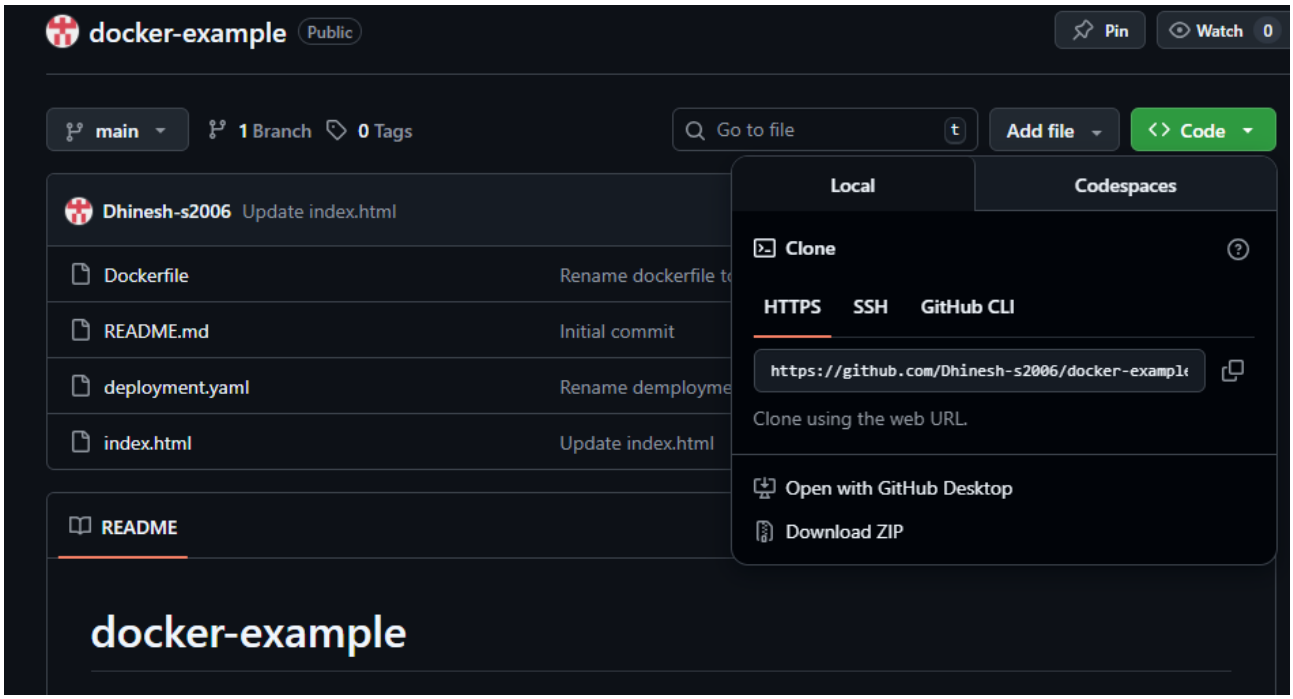
```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: html-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: html-demo
  template:
    metadata:
      labels:
        app: html-demo
    spec:
      containers:
        - name: html-demo
          image: dhinesh2006/html-demo:latest
          ports:
            - containerPort: 80

---
apiVersion: v1
kind: Service
metadata:
  name: html-demo-service
spec:
  type: NodePort
  selector:
    app: html-demo
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30010
```

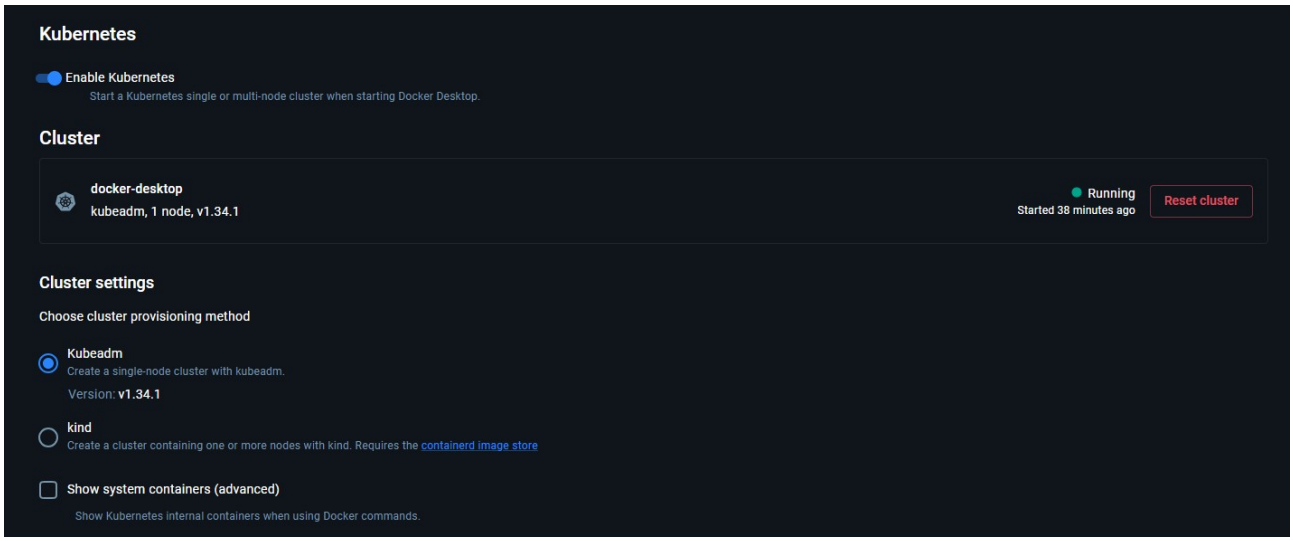
Dockerfile



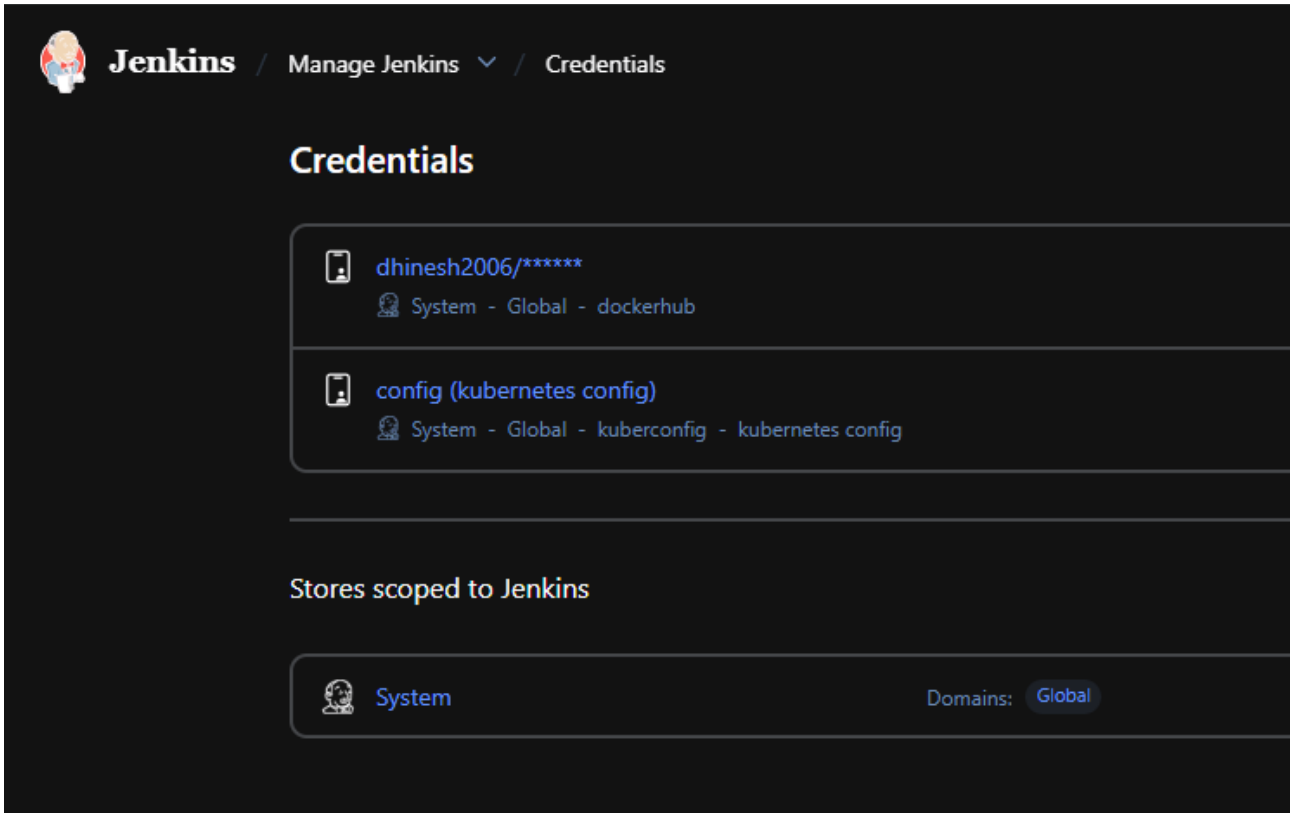
GitHub Repository



Enable Kubernetes



DockerHub Credentials and Kubernetes Config In Jenkins



Cloning the repository into VS Code and making updates and Pushing the changes to the Repository:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Batch1\Desktop\Dhinesh\docker-example> git push origin main
info: please complete authentication in your browser...
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 24 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 326 bytes | 326.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Dhinesh-s2006/docker-example.git
   013b55e..403646d  main -> main
PS C:\Users\Batch1\Desktop\Dhinesh\docker-example>
```

Pipeline Script:

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script

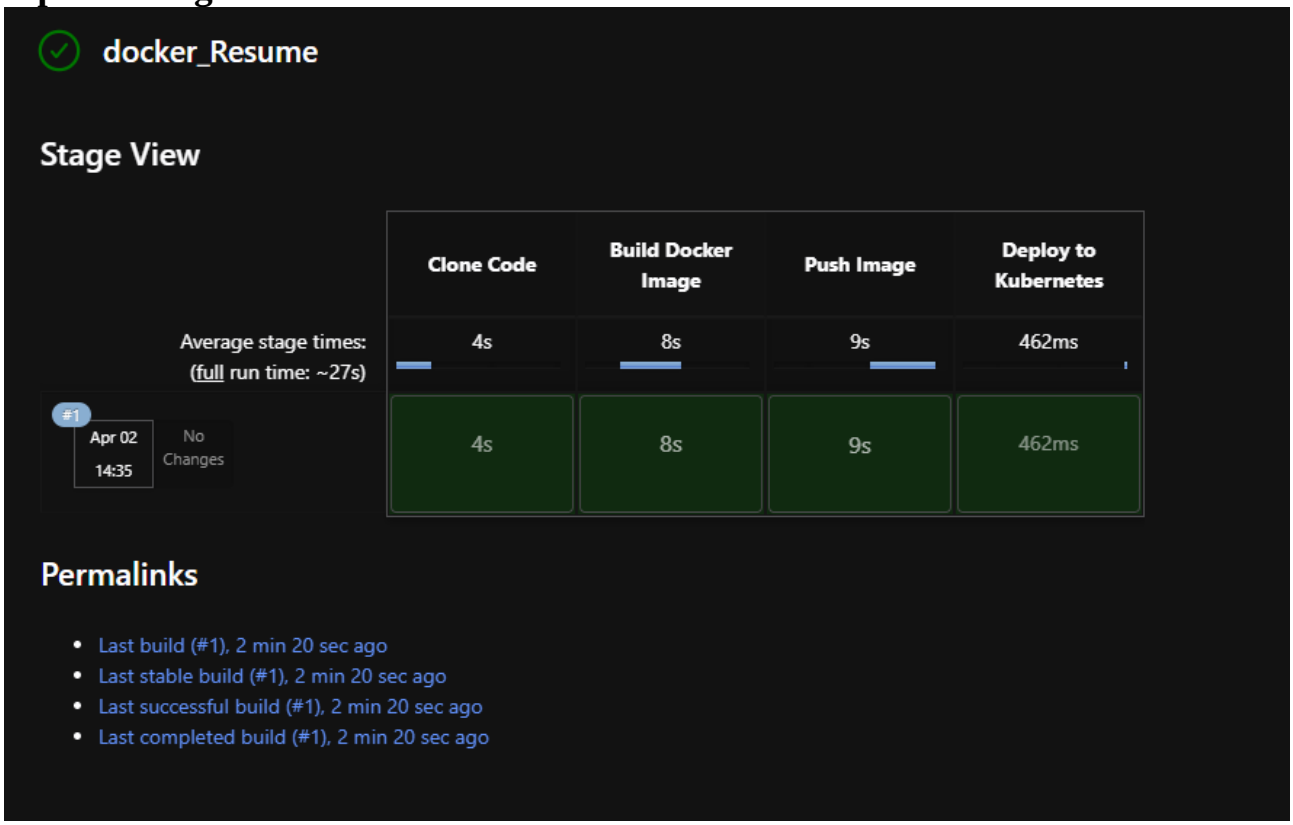
Script

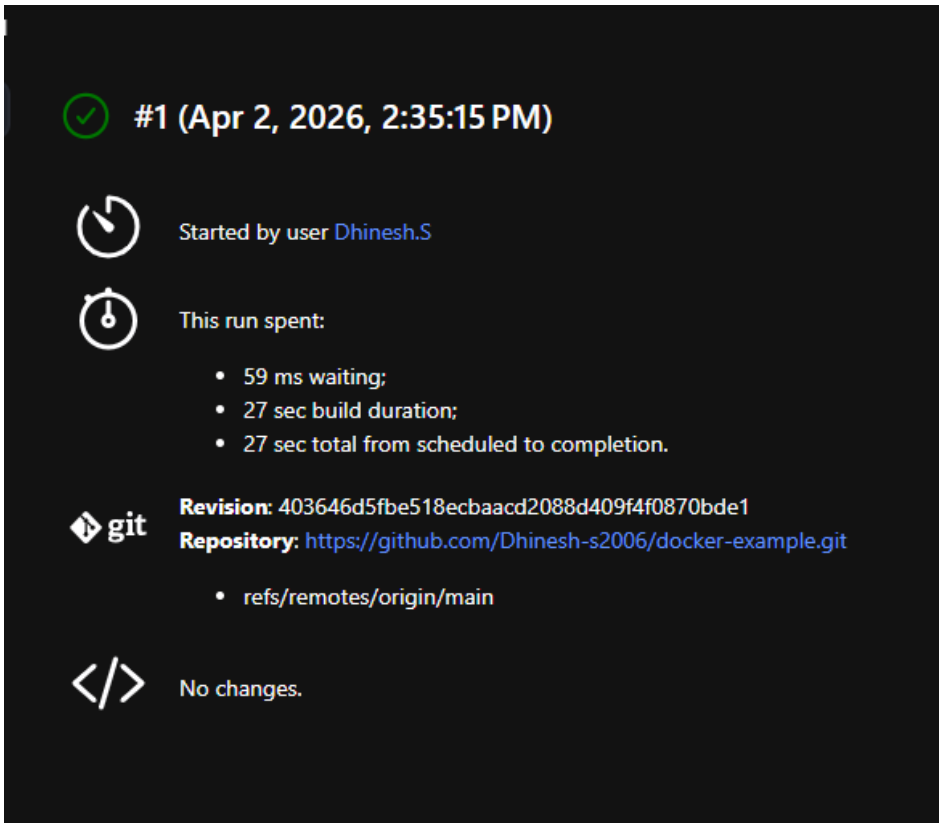
```
pipeline {
agent any
environment {
DOCKER_IMAGE = "dhinesh2006/html-demo"
}
stages {
 stage('Clone Code') {
steps {
git branch: 'main',
url: 'https://github.com/Dhinesh-s2006/docker-example.git'
}
}
stage('Build Docker Image') {
steps {
bat 'docker build -t %DOCKER_IMAGE% .'
```

☑ Use Groovy Sandbox

```
pipeline {
agent any
environment {
DOCKER_IMAGE = "dhinesh2006/html-demo"
}
stages {
 stage('Clone Code') {
steps {
git branch: 'main',
url: 'https://github.com/Dhinesh-s2006/docker-example.git'
}
}
stage('Build Docker Image') {
steps {
bat 'docker build -t %DOCKER_IMAGE% .'
}
}
stage('Push Image') {
steps {
withCredentials([usernamePassword(credentialsId: 'dockerhub',
usernameVariable: 'USER', passwordVariable: 'PASS')]) {
bat 'docker login -u %USER% -p %PASS%'
bat 'docker push %DOCKER_IMAGE%'
}
}
}
stage('Deploy to Kubernetes') {
steps {
withCredentials([file(credentialsId: 'kuberconfig', variable:
'KUBECONFIG')]) {
bat '''
set KUBECONFIG=%KUBECONFIG%
kubectl apply -f deployment.yaml --validate=false
'''
}
}
}
}
}
```

Pipeline Stage View:





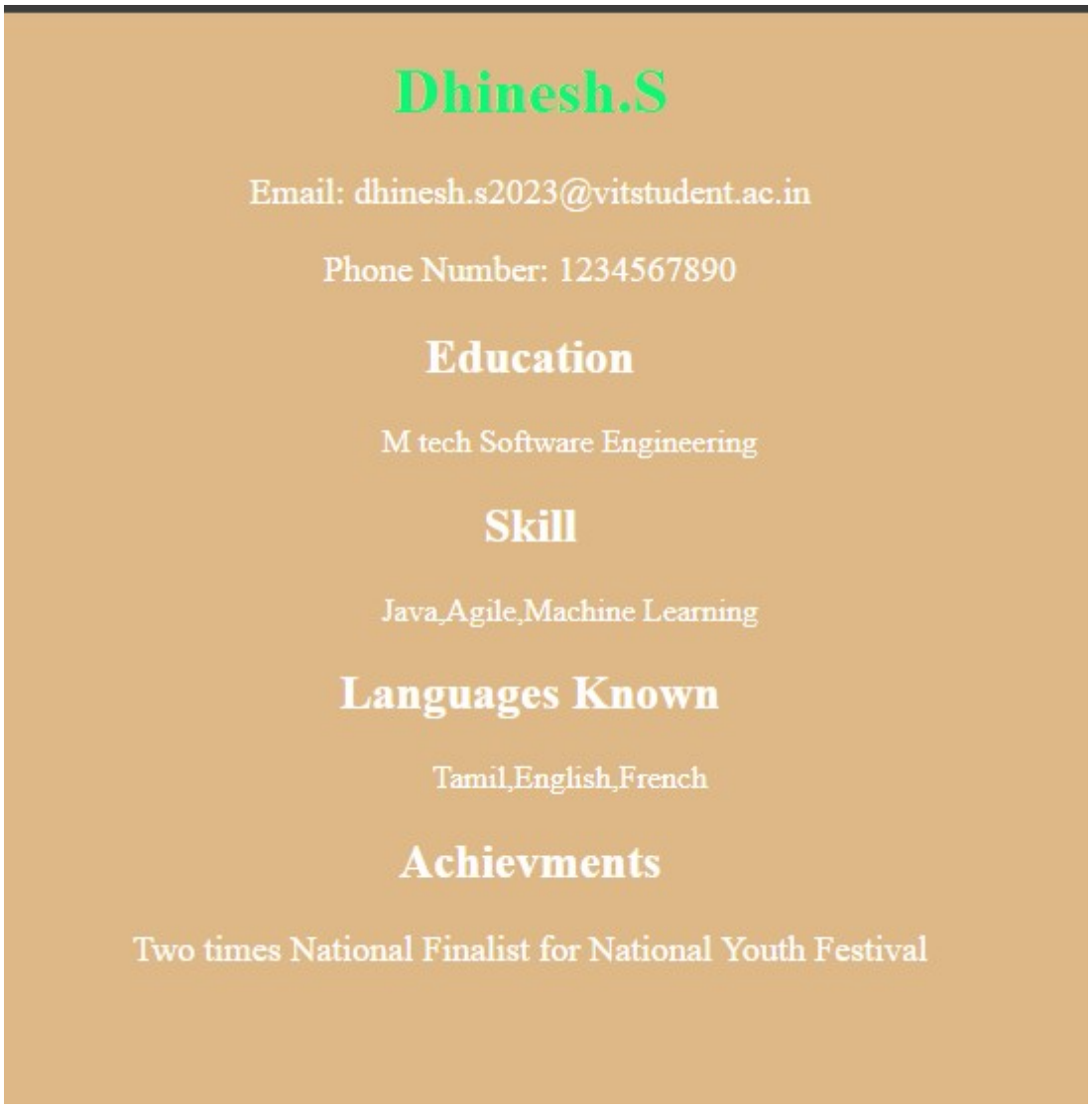
Console Output:

Console Output

```
e642839411b3: Waiting
5e815e07e569: Layer already exists
eb84b6c92ba4: Waiting
e642839411b3: Pushed
eb84b6c92ba4: Pushed
latest: digest: sha256:9c800d3ed1b88ecdeea607fe53ab2771e2aaa462495829dcf6095a6bca1762dc size: 856
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Deploy to Kubernetes)
[Pipeline] withCredentials
Masking supported pattern matches of %KUBECONFIG%
[Pipeline] {
[Pipeline] bat

C:\ProgramData\Jenkins\.jenkins\workspace\docker_Resume>set KUBECONFIG=****

C:\ProgramData\Jenkins\.jenkins\workspace\docker_Resume>kubectl apply -f deployment.yaml --validate=false
deployment.apps/html-demo created
service/html-demo-service created
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

Output

The application was successfully deployed and accessed using:http://localhost:30010